1. 서버 마이그레이션

온프레미스·타 클라우드의 서버를 SCP Virtual Server 로 이전하는 방식 2가지

구분	Migration Image	VM Migration
방식	VM 이미지(OVA)를 변환해서 서버 생성 (Cold)	가동 중인 서버를 실시간 복제 (Online)
도구	Object Storage + Migration Image 서비스	ZConverter (Agent + ZCM 포털)
서버 중지	필요	불필요
대상	VMware ESXi VM (가상 하드웨어 버전 19 이하)	가상 서버 + 물리 서버
특징	특정 시점 이미지 기준	Migration Image 대비 데이터 손실 기간이 짧음

Migration Image 절차: VM 을 OVA 로 Export → Object Storage 업로드 → Migration Image 로 변환 → Virtual Server 생성

Migration Image 요구사항

- 원본 서버에 VMware Tools 설치 (Linux 는 perl 도 설치)
- RDP/SSH 원격 접속 설정, 방화벽 설정
- RHEL 8, CentOS 8 이상은 Secure Boot 비활성화
- OS 디스크는 SCSI Index 0 에 연결
- 용량 제한: OVA 1TB, OS 디스크 200GB, 전체 디스크 합 1TB, 연결 디스크 7개
- 디스크 암호화 해제, 다중 네트워크 인터페이스 미지원
- USB·CDROM 연결 해제. Passthrough 디바이스, 과도한 CPU·Memory 사용 VM(DPDK, Hugepage)은 미지원

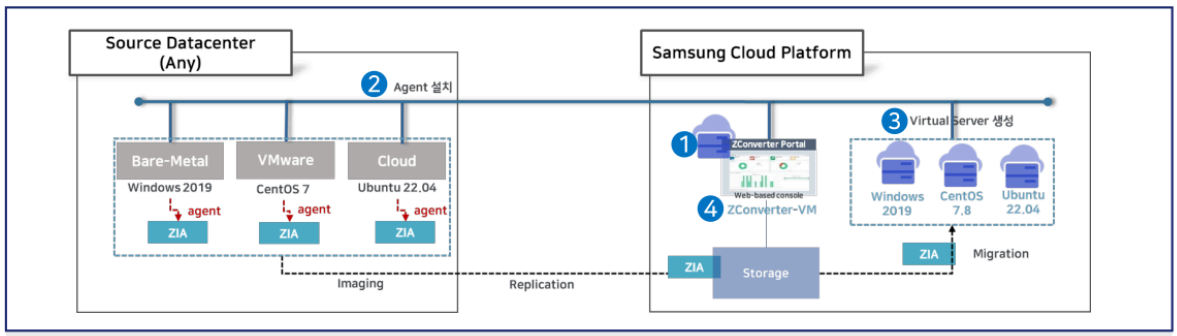
VM Migration 절차: SCP 에 Virtual Server 구성 → 원본 서버에 Agent 설치 → 대상 Virtual Server 생성 후 Agent 설치 → ZCM 포털 마이그레이션 마법사로 실행

2 Lab Overview (이론)

Samsung Cloud Platform

VM Migration 서비스

Migration Tool을 사용하여 On-Premise 가상 머신을 Samsung Cloud Platform으로 복제



1. Samsung Cloud Platform에 Virtual Server 구성
2. On-Premise/他 Cloud 서버(Source)에 Agent 설치
3. Samsung Cloud Platform에 Virtual Server 생성 및 Agent 설치
4. 마이그레이션 마법사를 통해 마이그레이션 수행(using ZCM Portal)

개념 - Cold Migration vs Online Migration

- Cold: 서버를 멈춘 시점의 이미지를 옮김. 이전 이후 생긴 변경분(예: 신규 등록 파일)은 반영되지 않음
- Online: 서비스를 유지한 채 복제. 변경 데이터 손실을 줄여야 할 때 사용
- Migration Image 사용처: 재설치가 불가능한 애플리케이션 VM, Virtual Appliance, 대량 VM 단기 이전, 하이브리드·DR 용 서버 중복 구성

2. 스토리지 마이그레이션

원본	대상	도구	비고
Azure Blob	Object Storage	rclone	복제용 Virtual Server 에 rclone 설치
Amazon EFS	File Storage	rsync	VPN 으로 EFS 를 Virtual Server 에 마운트 후 복사

- 사용 사례: 공유 스토리지의 대량 파일 이전, 대용량 미디어 콘텐츠 이전, 형식이 다른 객체 이전
- 이전 작업은 SCP 안의 Virtual Server 가 중간에서 수행
- EFS 는 NFS(2049) 포트가 필요하므로 Firewall / Security Group 에 해당 통신 허용

3. IPsec VPN 구성

목적

- 회선 비용 없이 인터넷 구간 트래픽 보호
- Virtual Server 를 온프레미스 네트워크로 안전하게 연결, 온프레미스 네트워크를 SCP 로 확장
- 제3자에게 노출되지 않는 데이터 전송

구성 순서: 로컬 Subnet 을 Virtual Server 에 연결 → VPN 생성 → VPN Tunnel 생성 → 고객 측 장비(예: Strongswan)를 SCP 에서 만든 VPN 설정값에 맞춤

IPsec / IKE 용어

용어	의미
Encryption algorithm	전송 데이터를 볼 수 없게 하는 암호화 알고리즘
Digest algorithm	데이터 무결성 확인용 해시 알고리즘
Diffie-Hellman	IKE 키의 안전성. 그룹 번호가 클수록 안전하지만 키 계산이 느림
PFS group	Perfect Forward Secrecy. 세션 키 보호를 위해 키를 교체해서 사용
SA lifetime	연결 완료부터 만료까지의 시간(초)
DF bit	Don't Fragment. 패킷 단편화 여부 설정 (Clear = 단편화, Copy = 유지)
DPD probe interval	Dead Peer Detection. 연결 중단 감지 주기(초)

> **개념 - IPsec 과 IKE**
>
> - IPsec: IP 계층에서 패킷 단위로 인증·암호화·키 관리를 수행하는 프로토콜
> - IKE: IPsec 의 키 관리 프로토콜. 보안 연결(SA) 생성·유지와 키 교환 담당
> - 양쪽 장비의 암호화·해시·DH 값이 일치해야 터널이 맺어짐

4. VPC Peering · Transit Gateway

구분	VPC Peering	Transit Gateway
연결	VPC 1:1 프라이빗 연결	여러 VPC, 여러 온프레미스 네트워크를 한 허브로 연결
경로	Internet Gateway 를 거치지 않음	단일 중개점 경유
제약	양쪽 Subnet IP 대역 중첩 불가	연결하는 모든 네트워크의 IP 대역 중첩 불가

연결 후 설정

- Routing Table: 상대 VPC 의 Private IP 대역을 목적지로 등록
- Firewall: 출발지 = Local IP, 목적지 = 상대 IP 로 규칙 추가
- 라우팅과 방화벽 둘 다 설정해야 통신됨

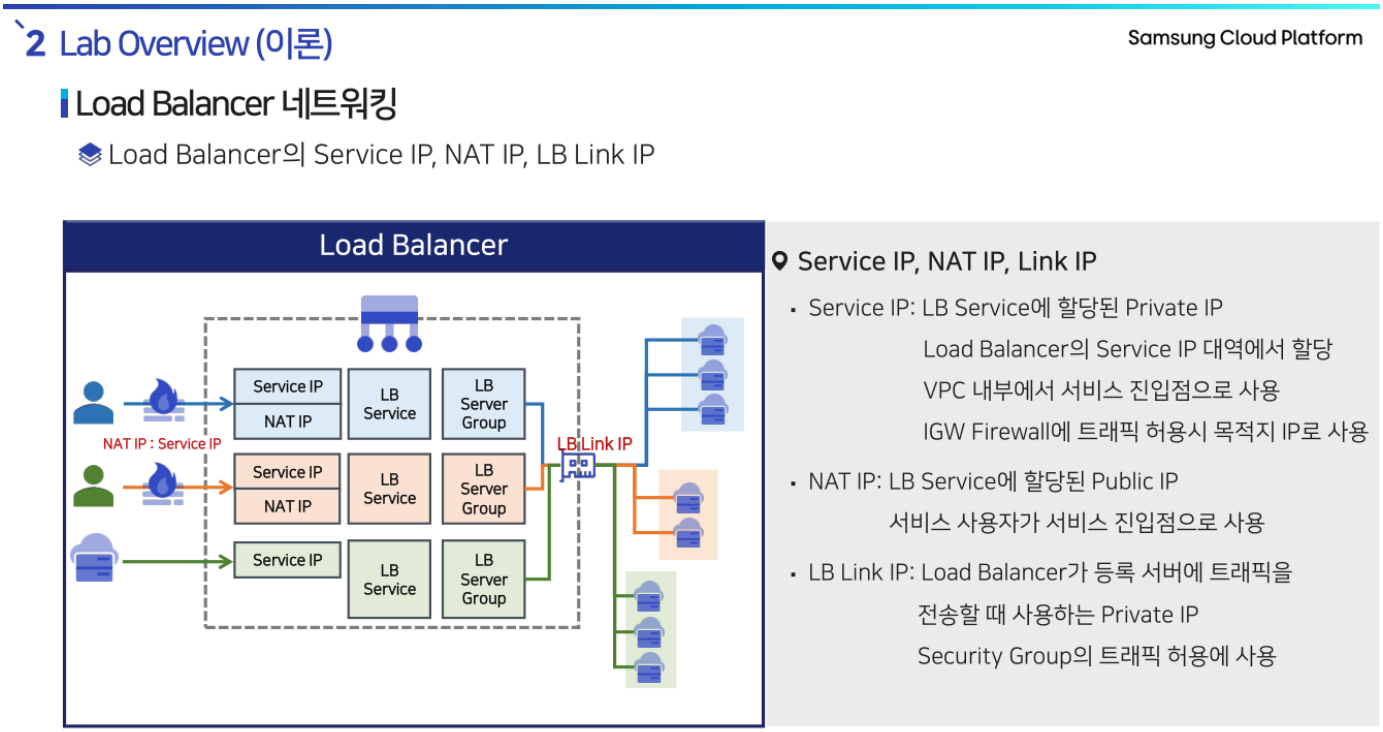
사용 사례

- VPC 별로 분리된 정보 시스템 간 안전한 통신
- 인터넷 경유 Outbound Traffic 비용 절감
- 한 VPC 의 Load Balancer 에 다른 VPC 의 Virtual Server 연결
- 다른 Region VPC 에 DB Read Replica 구성
- Transit Gateway: 여러 온프레미스 네트워크를 VPC 에 프라이빗 연결

5. 다중 VPC 간 Load Balancer

LB 의 3가지 IP

IP	설명	용도
Service IP	LB Service 에 할당된 Private IP (LB 서비스 대역에서 할당)	VPC 내부 서비스 진입점. IGW Firewall 허용 시 목적지 IP
NAT IP	LB Service 에 할당된 Public IP	외부 사용자의 서비스 진입점
LB Link IP	LB 가 등록 서버로 트래픽을 보낼 때 쓰는 Private IP	서버 Security Group 에서 허용



서버그룹에 등록 가능한 대상

- 같은 VPC 의 Virtual Server (서버 이름으로 등록), Auto-Scaling Group, Kubernetes Cluster
- VPC Peering 으로 연결된 다른 VPC 의 Virtual Server (Private IP 로 등록)

구성 순서: VPC Peering 연결 → 한쪽 VPC 에 LB 생성, LB Service 추가 → 각 서버 Security Group 에 LB Link IP 허용 규칙 추가 → 서버그룹에 서버 등록

연결 불가 대상

- Peering 으로 연결된 다른 VPC 의 Auto-Scaling Group, Kubernetes Cluster 는 서버그룹에 등록할 수 없음
- Transit Gateway 로 연결된 다른 VPC 의 서버도 등록 불가
- 서버가 LB Link IP 를 허용하지 않으면 Health Check 가 실패함

6. GSLB 를 이용한 부하 분산

GSLB 동작: 도메인 이름 해석(DNS) 요청에 대해 알고리즘에 따라 등록된 서버 IP 를 응답

항목	내용
분산 방식	Round Robin(순환, 균등 분배) / Ratio(IP 별 Weight 비율)
등록 서버	최대 8개
장애 조치	프로토콜(ICMP, HTTP 등)로 서버 상태 확인, 장애 서버는 응답에서 제외

사용 사례

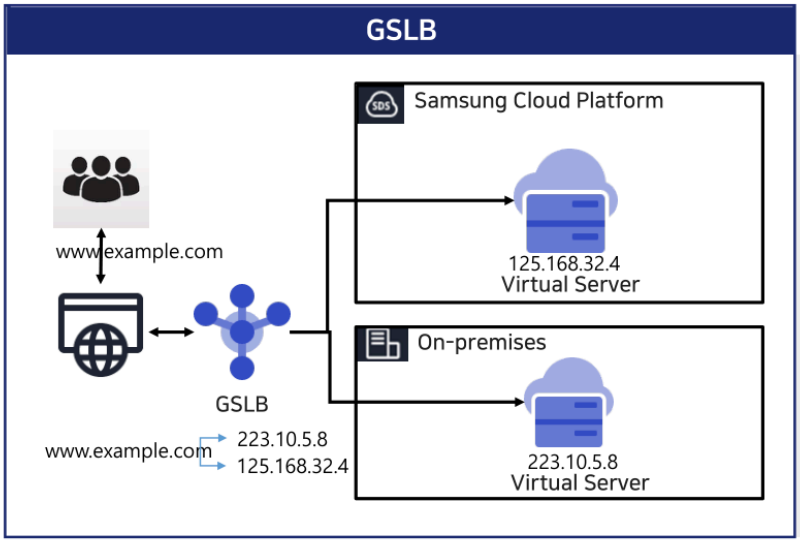
- 온프레미스 과부하 시 일부 요청을 클라우드로 분산 (하이브리드)
- 서버 장애·사이트 재해 시 보조 사이트로 요청 전환
- 여러 사이트에 사용자 요청 분산
- 신규 웹 애플리케이션 테스트 후 점진 전환 (Ratio 로 비율 조절)

2 Lab Overview (이론)

Samsung Cloud Platform

GSLB

GSLB는 특정 알고리즘을 통해 도메인 이름 해석 요청을 이용해 사용자 요청을 등록된 서버에게 순환 방식 또는 가중치 방식으로 응답 메시지를 전송합니다.



GSLB를 활용한 요청 분산

- Round Robin: 순환방식 기반으로 트래픽을 균등하게 분배하여, 도메인 응답 메시지로 전송
- Ratio: 연결 대상(IP)별 Weight 값에 비례하여, 도메인 응답 메시지로 전송
- 등록 서버 수: 최대 8개
- 장애 조치: 특정 프로토콜(ICMP, HTTP 등)을 지정하여 서버 상태를 체크하여 장애시 제외

개념 - GSLB vs Load Balancer

- LB: 한 VPC·Region 안의 서버로 트래픽을 직접 전달
- GSLB: DNS 응답 단계에서 사이트(Region, 온프레미스)를 고름. 트래픽이 GSLB 를 거치지 않고 응답받은 IP 로 바로 접속
- 사이트 단위 분산·장애 전환에는 GSLB, 서버 단위 분산에는 LB

7. 고가용성 3-Tier 아키텍처

목표: 단일 장애 지점(SPOF)을 제거하고 Downtime 을 최소화. 서비스 중단이 없어야 하는 중요 웹 서비스에 적용

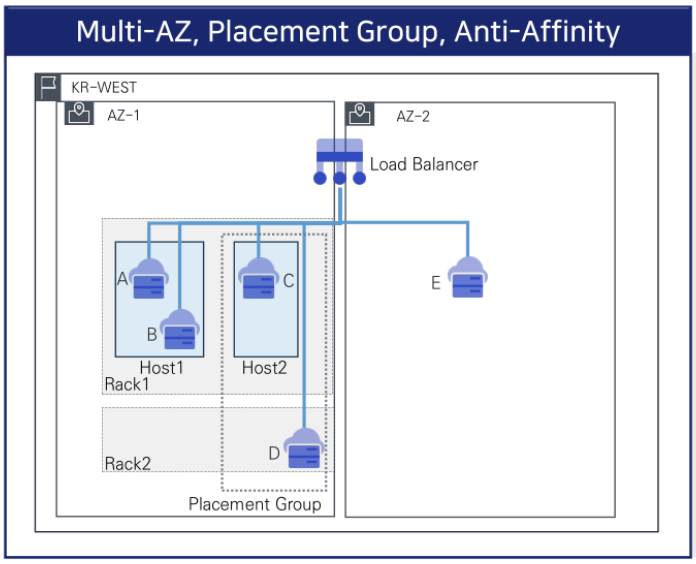
기능	분산 기준	효과
Placement Group	서로 다른 Rack 에 분산 배치	랙 단위 장애 방지
Anti-Affinity	같은 서버 그룹의 서버를 서로 다른 Host 에 배치	호스트 장애 시 전체 장애 방지
Multi-AZ	서로 다른 AZ 에 분산	데이터센터 장애 방지

2 Lab Overview (이론)

Samsung Cloud Platform

Samsung Cloud Platform의 서버 고가용성 제공 기능

Anti-Affinity, Placement Group, Multi-AZ



Multi-AZ

- 서로로부터의 장애로부터 격리되어 있고, 프라이빗 네트워크로 연결된 영역

Placement Group

- 관련 있는 자원들을 다른 랙으로 분산 배치하여 안정적으로 사용할 수 있게 함

Anti-Affinity

- 동일 서버 그룹의 가상 서버들을 다른 호스트에 배치해 호스트 문제 발생 시 모든 서버에 장애가 발생하는 위험 방지

구성 방법

- Web / App 서버를 Custom Image 로 만들어 같은 구성의 서버를 추가 생성
- Multi-AZ + Anti-Affinity 로 서버 생성, Placement Group 적용
- Web Tier, App Tier 앞에 Load Balancer 구성
- DB Tier 는 DBaaS HA(Active-Standby) 구성

> **개념 - SPOF**
>
> 한 곳이 멈추면 전체가 멈추는 지점. 서버 1대, DB 1대, AZ 1곳이 대표적. 각 계층(Web, App, DB)을 이중화 + 분산 배치해서 제거

8. 공유 스토리지와 Object Storage 콘텐츠 저장

서버에 파일을 저장하면 생기는 문제: 해당 서버가 정지하면 파일 접근 불가, 서버가 늘어나면 서버마다 파일이 달라짐

공유 스토리지	특징
Block Storage (Multi-Attach)	한 볼륨을 여러 서버에 연결
File Storage	Multi-AZ 생성 가능. 여러 Virtual Server 마운트. Kubernetes Engine 의 영구 볼륨(Persistent Volume)으로 사용 가능
Object Storage	REST API 로 저장·공유. 정적 콘텐츠, 대용량 파일

Auto-Scaling 과 공유 스토리지

- Auto-Scaling 서버에는 File Storage, Block Storage 공유 볼륨 연결 불가
- 콘텐츠를 공유하려면 Object Storage 사용

Object Storage 로 콘텐츠 이전

- 대상: 정적 콘텐츠(이미지, 동영상), 파일시스템에서 처리할 수 없는 데이터, 대용량 파일
- 절차: 기존 File Storage 의 콘텐츠를 Object Storage 로 복사 → 웹 애플리케이션 코드에서 로컬 경로를 Object Storage URL 로 수정
- 이전 도구: AWS CLI (S3 API 호환)
- 효과: 웹 서버 부하 감소, 서버 수가 늘어도 같은 콘텐츠 제공

9. DBaaS 고가용성 · 성능

HA 구성

- Active-Standby 이중화. 디스크 기반 동기(sync) 복제
- 장애 시 VIP 전환(VIP Switch)으로 1분 이내 Fail-Over
- 모든 관리형 RDB 에서 제공 (오픈소스 기반)

읽기 성능 향상

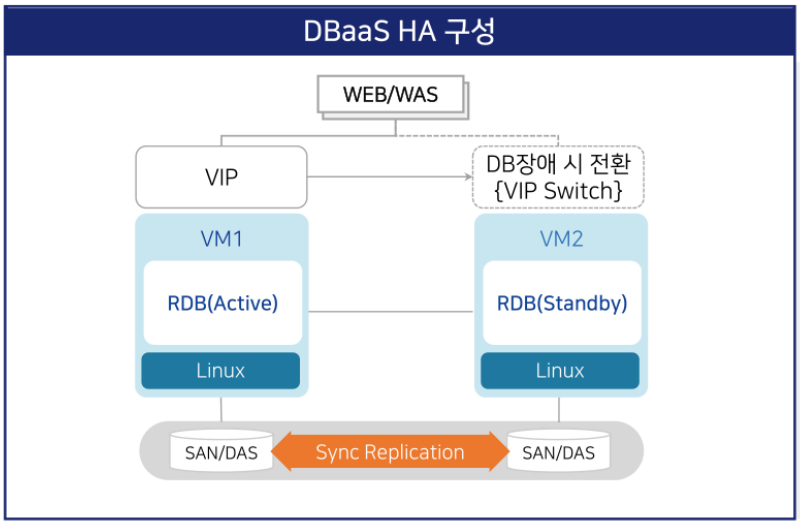
방법	설명
Read Replica	애플리케이션의 읽기 부하를 Replica 로 지정. 원본 DB 부하 감소. 수동으로 원본으로 승격 가능
Redis 캐싱	자주 읽는 데이터를 메모리에 두고 DB 조회 횟수를 줄임

2 Lab Overview (이론)

Samsung Cloud Platform

DBaaS의 고가용성

Active - Standby 이중화로 데이터베이스를 구성하여 1분 이내 장애 조치를 수행합니다.



사용 사례

- Active/Standby 이중화 구성
- 디스크 기반 동기(sync) 복제 수행
- Failover 시간 최소화

HA 와 Replica 역할 구분

- HA Standby: 장애 대비 전용. 평소에는 읽기·쓰기 처리 안 함
- Read Replica: 읽기 분산용. 쓰기는 Active DB 에서만 처리
- 고성능 DB 구성 = HA(가용성) + Read Replica(읽기 분산) + Redis(캐시)

10. Auto-Scaling 과 세션 관리

세션 관리가 필요한 이유

- 서버가 늘어나면 LB(Round Robin)가 요청을 여러 서버로 나눔
- 세션이 서버 메모리에 있으면 다른 서버로 가는 순간 로그인·장바구니가 사라짐
- 세션을 서버 밖(외부 저장소)에 저장해서 해결

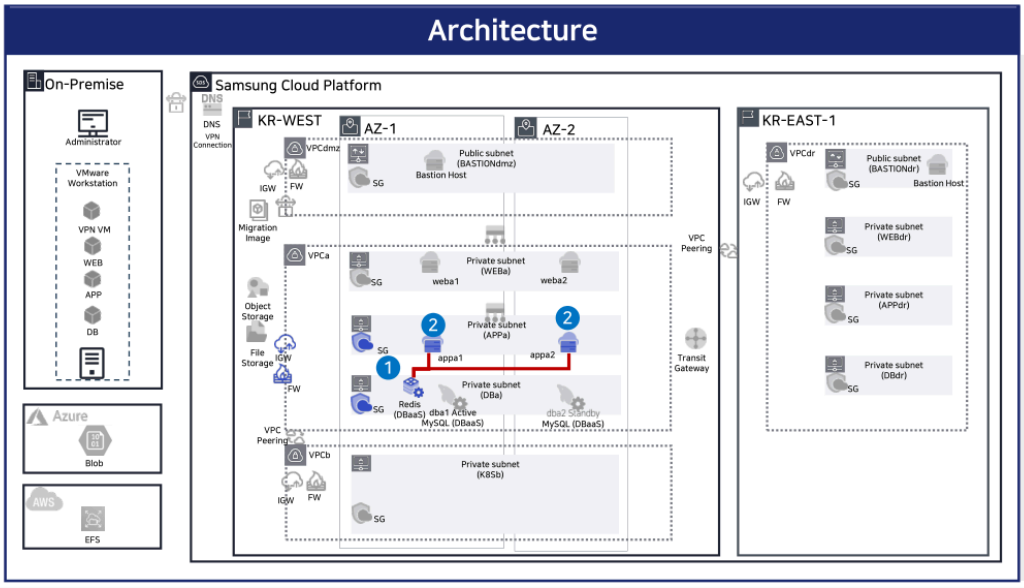
저장소	특징
Redis (DBaaS)	In-Memory DB. 빠른 처리. 관리형
Cassandra (Installed DB)	Virtual Server 에 직접 구축. 처리 용량을 사용자가 관리

2 Lab Overview (이론)

Samsung Cloud Platform

구현 아키텍처

이번 차시에서는 Auto-Scaling을 구현하기 위해 Session Management를 구성합니다.



1 Redis(DBaaS)를 생성합니다.

2 APP서버의 Session을 Redis에 저장할 수 있도록 변경합니다.

Auto-Scaling 정책

정책	설명
Scale-Out	지표가 임계치를 넘으면 서버 추가
Scale-In	부하가 줄면 서버 삭제
Schedule	지정한 주기·시간대·기간에 서버 수 조정 (시작일/종료일, 실행 주기)

- 판단 지표: CPU / Memory 사용률(%), Network In/Out, Disk Read/Write, Disk Read/Write Operation
- 증가 방식: 수량 증가 / 비율 증가
- 적정 임계값: 일정 기간 측정한 결과를 기준으로 설정. 서버가 필요 이상으로 늘거나 부하에 늦게 대응하는 문제 방지
- Group 설정: Min / Desired / Max 서버 수

> **개념 - 무상태(Stateless) 서버**
>
> 서버에 상태(세션, 업로드 파일)를 남기지 않는 구조. 서버를 언제든 추가·삭제할 수 있어야 Auto-Scaling 이 의미 있음. 세션 → Redis, 파일 → Object Storage 로 분리

11. Global CDN

동작: 전 세계에 분산된 엣지 서버가 원본 콘텐츠를 캐싱해서 사용자에게 가까운 곳에서 전달

기능	내용
캐싱	전송 정책과 캐시 만료 시간(TTL) 설정. 만료되면 엣지의 콘텐츠를 삭제(Purge)
원본(Origin)	웹 서버 또는 Object Storage
보안	원본 서버와 HTTPS 통신. DDoS, 웹 공격으로부터 콘텐츠 보호
효과	해외 사용자 지연 감소. 트래픽 폭주 시 원본 서버 부하 분산

구성 순서

1. Global CDN 생성: CDN 이름, 도메인 입력
2. 원본 설정: 도메인 또는 IP, 포트
3. 원본 경로 설정: Forward host header, Cache key hostname 을 Origin Hostname 으로 지정
4. 캐싱 설정
5. DNS 에 CDN 도메인을 CNAME 으로 등록
6. 무료 인증서를 발급받아 Certificate Manager 에 등록하면 HTTPS 적용

12. 백업 구성

대상	방법	복구
Virtual Server	Backup 상품에 서버 등록	백업본으로 서버 복원 (OS 형상 보존)
DBaaS	DBaaS 자체 백업 기능. data file + archive file, 스케줄·보존기간 설정	시점 복구. 새 DBaaS 로 복원
File Storage	Snapshot 스케줄(자동 생성)	스냅샷으로 복구, 파일 단위 복구
Object Storage	버전관리 설정 (변경 내역을 버전으로 기록)	이전 버전으로 객체 복구

백업 목적

- Virtual Server: 의도하지 않은 형상 변경 복원, 보안 침해로 손상된 OS 복구
- Database: 장애·오류로 인한 손상 복구, 특정 시점 데이터 복원
- 보존 기간은 RPO 기준으로 설정

> **개념 - RPO / RTO**
>
> - RPO(Recovery Point Objective): 허용 가능한 데이터 손실 시점. RPO 1시간 = 최대 1시간 전 데이터까지 복구 가능해야 함. 백업 주기를 정하는 기준
> - RTO(Recovery Time Objective): 장애 후 서비스를 복구하기까지 허용 시간
> - 백업은 Region 내 데이터 보호, DR 은 Region 장애 대비

13. 재해복구(DR) 구성

Region 단위 장애에 대비해 데이터와 서비스를 다른 Region(KR-WEST ↔ KR-EAST-1)에 복제

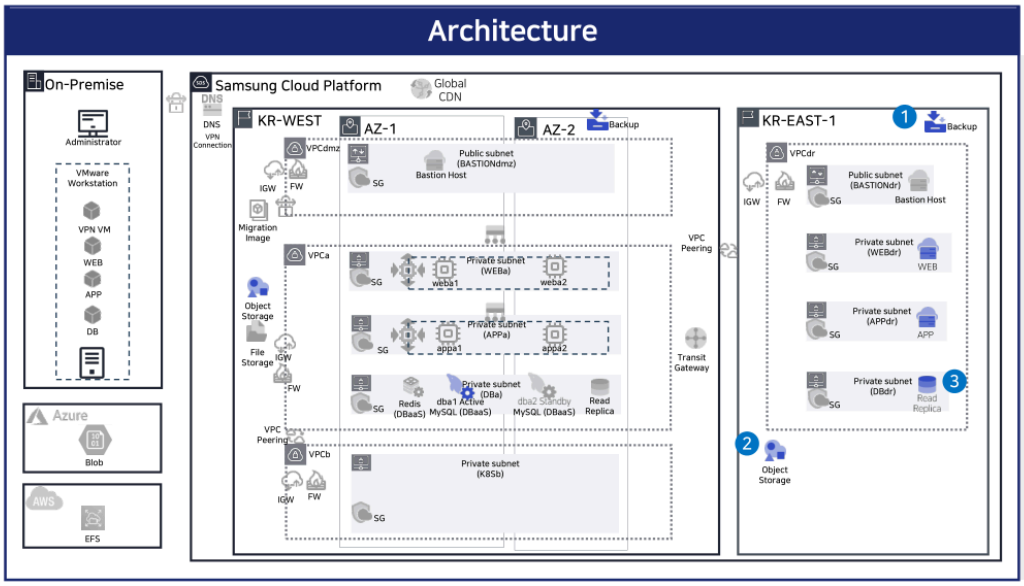
대상	DR 방법
Virtual Server	Backup Service 생성 시 DR 옵션 지정 → 양쪽 Region 에 백업본 복제 → 백업본으로 서버 생성
Object Storage	양쪽 Region 에 Bucket 생성 → 양쪽 모두 버전관리 활성화 → 원본 Bucket 에서 DR 동기화 설정
DBaaS	원본에서 Other-Region 설정으로 Read Replica 생성. DB 배포 후에도 설정 가능
트래픽 전환	GSLB 로 사이트 장애 시 DR 사이트로 전환

2 Lab Overview (이론)

Samsung Cloud Platform

구현 아키텍처

이번 차시에서는 DR을 위한 스토리지/데이터베이스 복제를 수행합니다.



1 DR 활성화로 Backup Service를 생성합니다.

2 KR-EAST-1에 Object Storage Bucket을 생성하고, 복제를 구성합니다.

3 MySQL(DBaaS)의 Read Replica를 KR-EAST-1에 생성하여 데이터를 복제합니다.

DR 사전 조건

- Object Storage DR 동기화는 양쪽 Bucket 모두 버전관리가 켜져 있어야 함
- DBaaS Other-Region Replica 는 양쪽 Region VPC 간 VPC Peering 과 라우팅 설정이 필요
- DR Region 에 VPC, Subnet 등 기본 네트워크를 미리 구성

14. 컨테이너와 Kubernetes

배포 방식 변화: 물리 서버(자원 활용 제한) → 가상 머신(OS 포함 전체 실행) → 컨테이너(OS 공유, 가볍고 이식성 높음)

컨테이너 장점: 빠른 배포, 지속적 통합·배포(CI/CD), 개발·운영 환경 일관성, 클라우드·OS 간 이식성, 마이크로서비스에 적합

Kubernetes 제공 기능: 서비스 디스커버리와 로드밸런싱, 스토리지 오케스트레이션, 자동 롤아웃·롤백, 자동 빈패킹, 자동 복구(self-healing), 시크릿·구성 관리

구성 요소

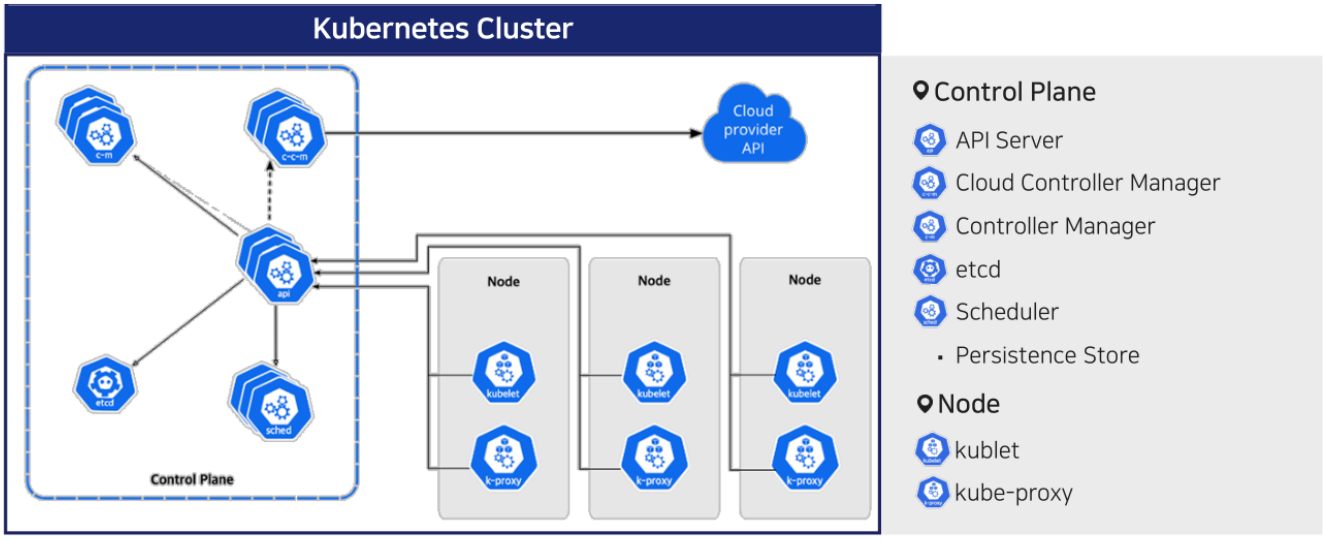
영역	구성 요소	역할
Control Plane	kube-apiserver	모든 요청 수신, 인증·검증, etcd 저장·갱신. Control Plane 의 Front-end
Control Plane	kube-scheduler	노드가 없는 Pod 에 노드 배정. 필터링 → 스코어링. 적합한 노드가 없으면 Pending
Control Plane	kube-controller-manager	리소스의 현재 상태를 정상 상태에 맞춤 (노드, 잡, 서비스어카운트 컨트롤러 등)
Control Plane	cloud-controller-manager	클러스터를 클라우드 공급자 API 에 연결
Control Plane	etcd	일관성이 강한 분산 키-값 저장소. 클러스터 상태 저장
Node	kubelet	노드의 Agent. Pod 생명주기 관리, 노드·Pod 모니터링
Node	kube-proxy	노드 네트워크 규칙 유지. 내부 통신, 외부 → Pod 통신
Node	Container Runtime	CRI 지원 런타임 (containerd, CRI-O, Docker Engine 등)

2 Lab Overview (이론)

Samsung Cloud Platform

Kubernetes Components

Kubernetes Architecture는 Control Plane과 Node로 구성됩니다.



개념 - Control Plane 과 Node

- Control Plane: 클러스터를 "결정"하는 쪽. 스케줄링, 상태 관리, 설정 저장
- Node: 컨테이너(Pod)가 "실행"되는 서버. 컨테이너 묶음 단위가 Pod
- 선언한 상태(원하는 Pod 수)와 현재 상태를 계속 맞추는 것이 Kubernetes 의 기본 동작

15. Container Registry 와 배포

Container Registry: 컨테이너 이미지와 OCI(Open Container Initiative) 표준 아티팩트를 저장·관리·공유하는 클라우드 레지스트리

기능	내용
Registry 관리	생성, 삭제, 접근 제어
이미지 관리	태그 관리(Push / Pull / 조회 / 삭제), 리포지토리 관리(보안 정책 설정)
취약점 점검	CVE 기반 스캐닝. 대상: 컨테이너 이미지, 패키지, Secret, Language Package

Kubernetes Apps: Kubernetes 클러스터에 소프트웨어를 컨테이너 이미지로 바로 배포하는 서비스. Ingress Controller 를 배포해서 Load Balancer 와 연결

배포 흐름

1. Kubernetes Client(Bastion Host)에서 이미지 빌드
2. Container Registry 에 이미지 저장, 버전(태그) 관리
3. Registry 의 이미지를 Kubernetes Cluster 에 컨테이너로 배포
4. Ingress Controller + Load Balancer 로 외부 서비스

16. Serverless 구성

Serverless: 서버 프로비저닝 없이 관리형 서비스로 컴퓨팅 수행

방식	사용자 관리 범위
Physical Machine	하드웨어부터 애플리케이션까지 전부
Virtual Machine	OS + Bin/Lib + 앱
Container	컨테이너 + 앱 (OS 공유)
Serverless	코드(함수)만. 하드웨어~Runtime 은 CSP 제공, 자동 스케일

SCP 서비스

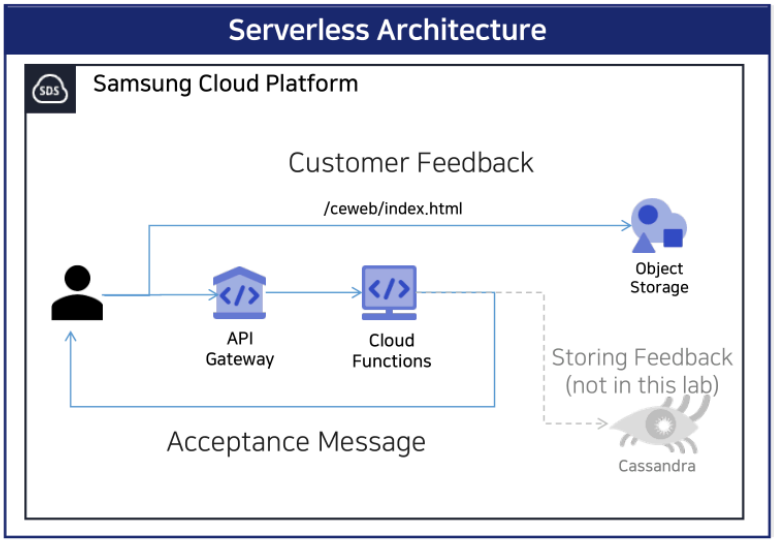
- Cloud Functions: FaaS. 이벤트 트리거로 실행. Runtime 은 Python, Node.js, PHP, Go 등. 엔드포인트·Token·접근 제어, 환경 변수 관리
- API Gateway: API Collection 단위 관리. 사용량 정책, 캐시, Token 발급, IP White List, Swagger Import, 호출 현황·응답 시간 조회
- Cloud Functions 에 요청을 전달하려면 API Gateway 구성 필요
- Object Storage: 정적 웹 사이트 호스팅 (폴더 단위 Public 권한, AES-256 기본 암호화, 버전관리)

2 Lab Overview (이론)

Samsung Cloud Platform

Serverless Architecture 구현

서버를 사용하지 않고 전체 기능 흐름을 구현하는 Serverless Architecture



구현 시나리오

Object Storage의 정적 웹페이지에서 제출된 요청은
API Gateway의 라우팅을 통해
Cloud Functions 로 전달됩니다.

- Object Storage — 정적 웹 사이트
- API Gateway — 사용자 피드백 라우팅
- Cloud Functions — 피드백 접수 (및 저장)
- User — 피드백 접수 메시지 수신

개념 - Serverless 흐름 예시

정적 웹 페이지(Object Storage)에서 사용자가 피드백 제출 → API Gateway 가 라우팅 → Cloud Functions 가 접수 처리 → 필요 시 DB 저장. 평소 서버가 떠 있지 않고 호출될 때만 실행·과금되므로 비용 최적화에 사용

17. DevOps 개념과 도구

개발 방법론

- Waterfall: 요구사항 → 분석 → 설계 → 구현 → 테스트 → 유지보수 단계별 진행. 문서 중심, 변화 대처가 어려움
- Agile: 짧은 반복 주기(Sprint)로 개발·개선. 새 기능을 빠르게 전달

DevOps: Development + Operation. 개발팀과 운영팀이 협력해서 애플리케이션을 신속하게 제공하기 위한 문화, 철학, 방식, 도구의 조합. 지속적 통합·배포(Code → Build → Integrate → Release → Deploy)

DevOps Service 도구: SCP 에서 CI/CD 도구를 통합 관리

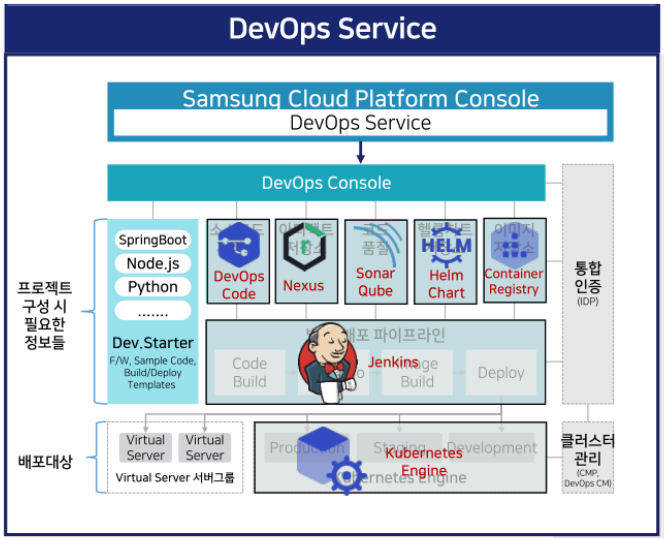
구분	도구	역할
소스 코드 저장소	DevOps Code	Private Git. Branch, Pull Request, Webhook, 개인 인증 토큰, Repository 별 권한(Admin / Write / Read)
빌드·배포 파이프라인	Jenkins	빌드, 테스트, 배포 작업 자동화 서버
아티팩트 저장소	Nexus	jar, war 등 빌드 산출물, 공통 라이브러리 버전 관리
코드 품질	SonarQube	버그, 취약점 탐지. 품질 게이트로 기준 설정
Helm Chart 저장소	Helm Chart Repository	Kubernetes 리소스 정의(Chart) 저장·공유
이미지 저장소	Container Registry	컨테이너 이미지 저장

2 Lab Overview (이론)

Samsung Cloud Platform

DevOps Service

Samsung Cloud Platform 에서 CI/CD 도구를 통합 관리해주는 서비스



DevOps 구성 도구

- 소스코드 저장소 — DevOps Code
- 아티팩트 저장소 — NEXUS
- 코드 품질 — SonarQube
- 헬름차트 저장소 — Helm Chart
- 이미지 저장소 — Container Registry
- 빌드/배포 파이프라인 — Jenkins

개념 - CI / CD

- CI(지속적 통합): 코드 변경을 저장소에 자주 합치고 빌드·테스트를 자동 실행
- CD(지속적 배포): 검증된 결과물을 자동으로 운영 환경까지 배포
- Webhook 으로 코드 변경 시 빌드를 자동 시작. Helm 은 Kubernetes 의 패키지 관리·배포 도구

18. DevOps Service 구성 흐름

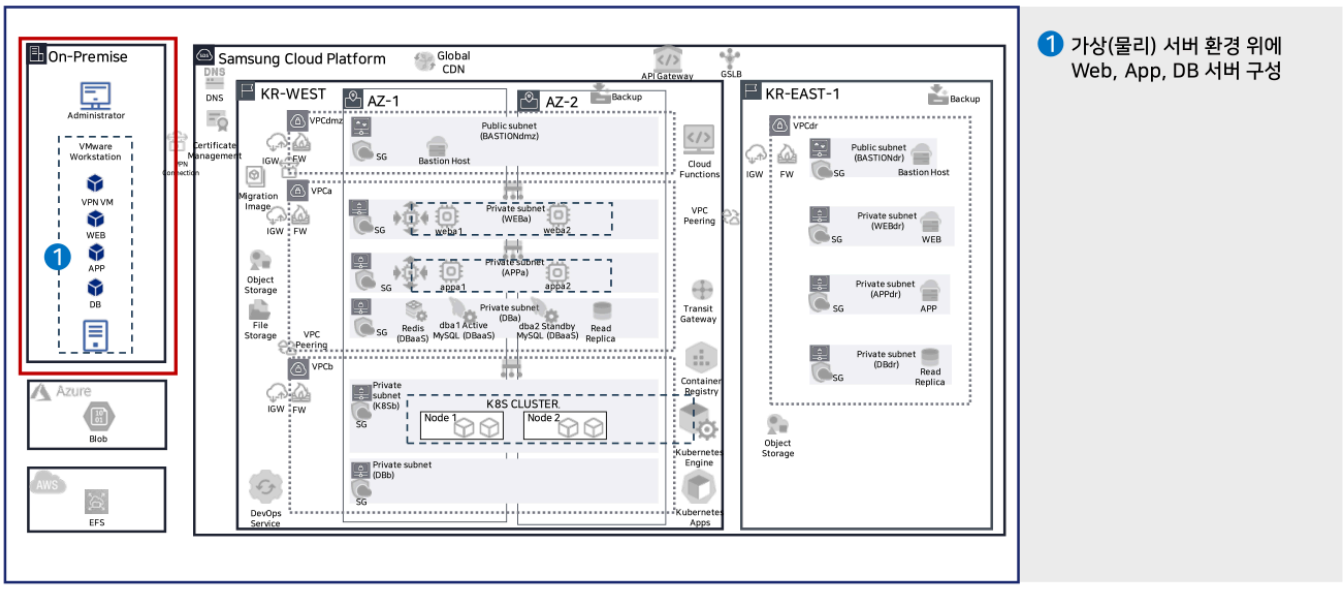
순서	작업
1	DevOps Service 생성
2	도구 등록: Jenkins, Nexus, SonarQube, Container Registry, DevOps Code
3	배포 대상 클러스터 등록: Kubernetes, Virtual Server
4	프로젝트 그룹 생성
5	프로젝트 생성: 프로젝트명, 배포 타겟 선택
6	프로젝트 설정: 애플리케이션 템플릿(런타임), 코드 저장소, 코드 품질, 배포 대상 환경, Helm 차트, 빌드 파이프라인, 이미지 저장소
7	배포 완료

- 도구 준비: Kubernetes Apps 에서 Jenkins, Nexus, SonarQube 를 배포하고 DevOps Code 생성
- 사전 구성: VPC, Internet Gateway, Subnet, NAT Gateway, Load Balancer, Security Group, Firewall, Object/File Storage, Container Registry, Kubernetes Engine, Ingress Controller
- 클러스터 등록 정보: Name Space, Node Selector, Ingress Domain
- 결과: 코드 작성 → 빌드 → 배포 → 서비스 개시까지 일관된 파이프라인 구축 (CI/CD)

19. Architecture Review

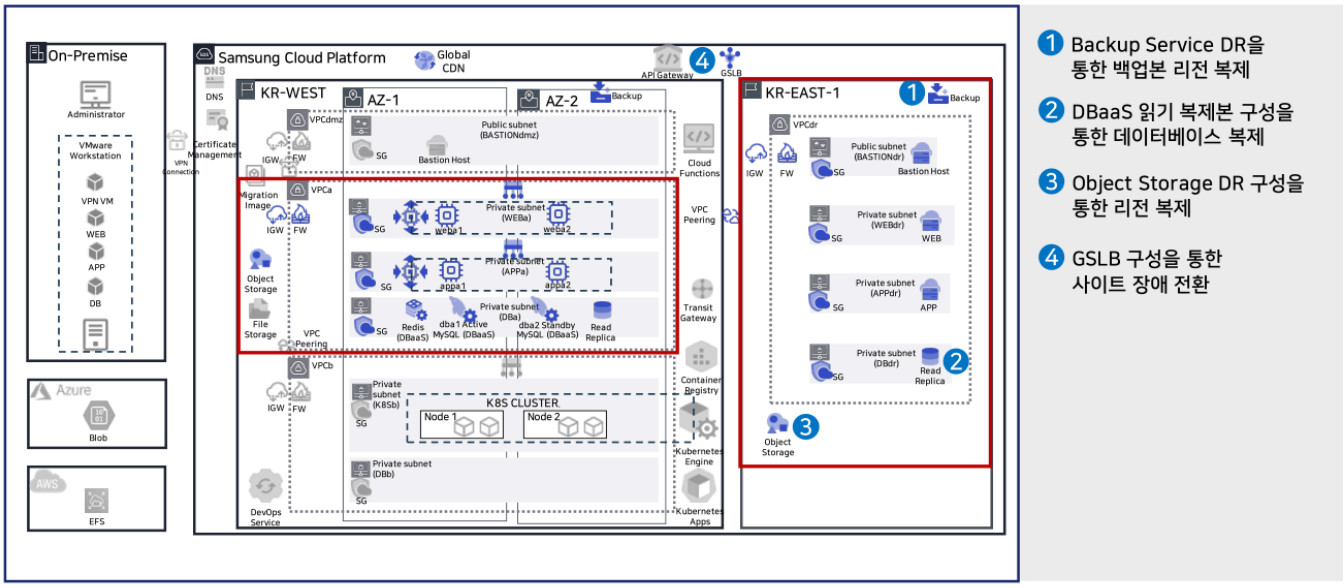
심화 과정 전체를 3가지 아키텍처로 정리

① 고가용성 Auto-Scaled 3-Tier



- Web·App Tier 에 Multi-AZ + Auto-Scaling 적용
- Web Tier 공유 스토리지는 Object Storage, Global CDN 연결
- DBaaS HA + 읽기 성능을 위한 Redis 캐싱, Read Replica

② 재해복구(DR)



- Backup Service DR 로 백업본 Region 복제

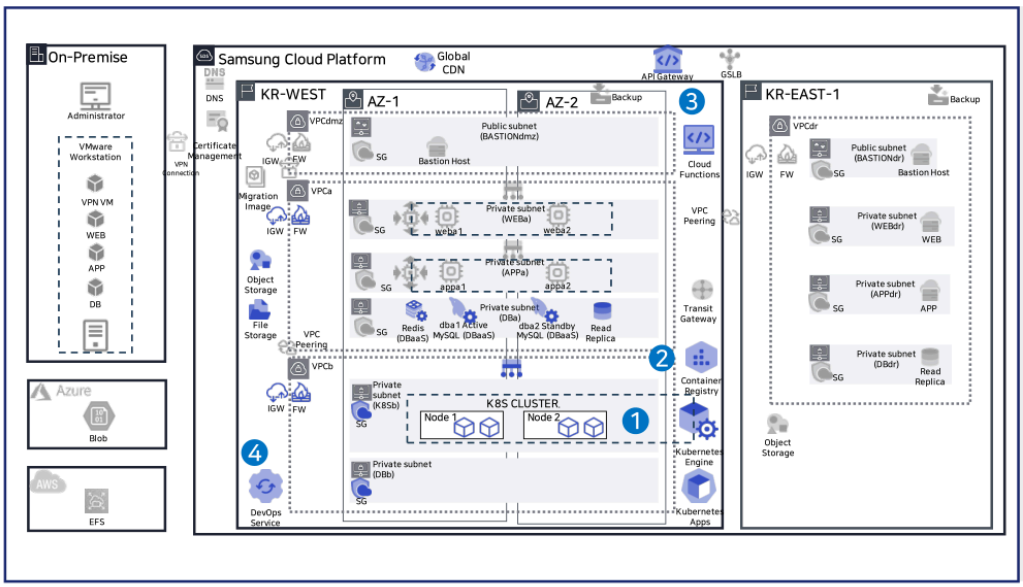
- DBaaS 읽기 복제본으로 DB 복제
- Object Storage DR 로 Region 복제
- GSLB 로 사이트 장애 전환

③ Cloud Native

1 Architecture Review

Samsung Cloud Platform

Cloud Native Architecture on Samsung Cloud Platform



1. Kubernetes Engine에서 애플리케이션 배포
2. Container Registry에 컨테이너 이미지 저장 및 버전 관리
3. API Gateway와 Cloud Functions를 통한 Serverless Computing 구현
4. DevOps Service 구성을 통한 CI/CD 구현

- Kubernetes Engine 에 애플리케이션 배포, Container Registry 로 이미지 버전 관리
- API Gateway + Cloud Functions 로 Serverless 구현
- DevOps Service 로 CI/CD 구현

요구사항별 서비스 정리

요구사항	서비스
서버 장애에도 서비스 유지	Multi-AZ, Anti-Affinity, Load Balancer
부하 증가 대응	Auto-Scaling + 세션 외부 저장(Redis)
DB 장애·읽기 부하	DBaaS HA, Read Replica, Redis 캐시
해외 사용자 지연	Global CDN
Region 장애	Backup DR, Object Storage DR, DB Other-Region Replica, GSLB
배포 자동화	DevOps Service (Jenkins, Nexus, SonarQube)